

Uyarı: Bu içerik, SCÜ Şarkışla UBYO Doğal Dil İşleme dersi kapsamında tamamen eğitim amaçlı çevrilmiş ve derlenmiştir. Orijinal dokümantasyon kaynakları (Hugging Face LLM Course, Transformers, Datasets, Tokenizers kütüphaneleri) kendi orijinal lisanslarına (Apache 2.0, MIT, CC-BY 4.0) tabidir. Bu çalışmanın hiçbir ticari amacı yoktur.

Sunum Notları · Bölüm 1: Transformer Modelleri

İçindekiler

- 1/1 · Giriş — Özge Sayınbaş
- 1/2 · Doğal Dil İşleme ve Büyük Dil Modelleri — Özge Sayınbaş
- 1/3 · Transformer'lar neler yapabilir? — Himmet Can Umutlu
- 1/4 · Transformer'lar nasıl çalışır? — Simay Evin
- 1/5 · Transformer'lar görevleri nasıl çözer? — Himmet Can Umutlu
- 1/6 · Transformer Mimarileri — Simay Evin
- 1/7 · Kısa sınav — Sıla Taşan
- 1/8 · LLM'lerle çıkarım — Himmet Can Umutlu
- 1/9 · Önyargı ve sınırlamalar — Abdulkadir Öcal
- 1/10 · Ünite Özeti — Özge Sayınbaş
- 1/11 · Sertifika sınavı — Sıla Taşan

1/1 · Giriş — Özge Sayınbaş

Bu bölüm, Hugging Face ekosistemindeki Transformers, Datasets, Tokenizers ve Accelerate kütüphaneleri ile Hugging Face Hub'ı kullanarak büyük dil modellerini (LLM) ve doğal dil işlemeyi (NLP) öğretir. İlk bölümde, metin üretimi ve sınıflandırma gibi NLP görevlerinin `pipeline()` fonksiyonuyla nasıl çözüldüğü, Transformer mimarisi ve encoder, decoder ve encoder-decoder mimarilerinin farkları ile kullanım alanları ele alınır.

Doğal Dil İşleme (NLP), bilgisayarların insan dilini anlamasını, yorumlamasını ve üretmesini sağlayan geniş bir alandır. Duygu analizi (sentiment analysis), adlandırılmış varlık tanıma (Named Entity Recognition, NER) ve makine çevirisi (machine translation) bu alanın görevlerine örnektir.

Örnek: Bir ürün yorumunun olumlu mu olumsuz mu olduğunu bulmak (duygu analizi) ya da bir cümleyi Türkçeden İngilizceye çevirmek (makine çevirisi) birer NLP görevidir.

Büyük Dil Modelleri (Large Language Models, LLM), NLP modellerinin güçlü bir alt kümesidir. Çok büyük boyutları ve çok büyük miktarda veriyle eğitilmeleri sayesinde, göreve özgü çok az eğitimle birçok farklı dil görevini yerine getirebilirler. GPT, Llama ve Claude bu modellere örnektir.

Örnek: Eskiden çeviri için bir model, özet çıkarmak için başka bir model gerekirdi. Bir LLM ise tek başına hem çeviri yapabilir hem özet çıkarabilir hem de soruları yanıtlayabilir.

Kısacası NLP geniş bir alandır; LLM ise bu alanın en güçlü araçlarından biridir. LLM'lerle verimli çalışabilmek için NLP'nin temel kavramlarını bilmek gerekir.

1/2 · Doğal Dil İşleme ve Büyük Dil Modelleri — Özge Sayınbaş

NLP, dilbilim ile makine öğrenmesinin kesişiminde yer alır ve kelimeleri bağlamlarıyla birlikte anlamayı amaçlar. Başlıca görevleri şunlardır:

- Cümleyi bütün olarak sınıflandırmak:** "Bu film harikaydı!" → olumlu yorum · "Bedava telefon kazandınız!" → spam
- Cümledeki kelimeleri sınıflandırmak:** "Ayşe Ankara'da çalışıyor." → Ayşe = kişi, Ankara = yer
- Metin üretmek:** "Bugün hava çok ..." → model istemi (prompt) tamamlar
- Metinden yanıt çıkarmak:** Metin: "Ali 2010'da doğdu." · Soru: "Ali ne zaman doğdu?" → 2010
- Yeni metin üretmek:** Çeviri ("Good morning" → "Günaydın") veya uzun bir metni kısa bir özete dönüştürmek

NLP yalnızca yazıyla sınırlı değildir; bir ses kaydını yazıya dökmek (konuşma tanıma, speech recognition) ve bir fotoğrafı betimlemek de bu alana girer.

LLM'ler, devasa metin verisiyle eğitilmiş ve göreve özgü eğitim olmadan birçok dil görevini yapabilen modellerdir. Milyarlarca parametreye sahiptirler, istemde verilen örneklerden öğrenebilirler (bağlam içi öğrenme, in-context learning) ve büyüdükçe açıkça programlanmamış ya da öngörülmemiş yetenekler gösterebilirler (ortaya çıkan yetenekler, emergent abilities).

Örnek (bağlam içi öğrenme): Modele "mutlu → olumlu, üzgün → olumsuz, sevinçli → ?" yazılırsa, model kalıbı anlayıp "olumlu" cevabını verir. Bunun için modeli yeniden eğitmeye gerek yoktur.

Sınırlamaları:

- Halüsinasyon (hallucination):** Yanlış bilgiyi emin bir şekilde üretebilir. *Örnek: Var olmayan bir kitabı gerçekmiş gibi anlatmak.*
- Gerçek anlama eksikliği:** Dünyayı gerçek anlamda kavrayamaz; tamamen istatistiksel örüntülerle çalışır.
- Önyargı (bias):** Eğitim verisindeki önyargıları tekrarlayabilir. *Örnek: Meslekleri belirli bir cinsiyetle eşleştirmek.*
- Sınırlı bağlam penceresi (context window):** Çok uzun bir metnin tamamını aynı anda dikkate alamaz.
- Yüksek hesaplama ihtiyacı:** Eğitmek ve çalıştırmak için güçlü donanım gerekir.

Dil işlemenin zorluğu: Bilgisayarlar metni insanlar gibi anlayamaz; metnin önce modelin işleyebileceği bir biçime dönüştürülmesi gerekir.

Örnek: "Acıktım" cümlesini okuyan biri anlamını hemen kavrar; "Acıktım" ile "Üzgünüm" cümlelerinin ne kadar benzediğini de kolayca söyler. Bir model için bu zordur.

Çok anlamlılık, kültürel bağlam, iğneleme (sarkazm) ve mizah, LLM'ler için bile hâlâ zor konulardır. *Örneğin yağmurda ıslanan birinin "Ne güzel hava!" demesindeki iğnelemeyi anlamak.*

1/3 · Transformer'lar neler yapabilir? — Himmet Can Umutlu

pipeline() Fonksiyonunun Çalışma Mantığı

- `pipeline()` Transformers kütüphanesinin en temel nesnesidir.
- Bir modeli, ön işleme (preprocessing) ve son işleme (postprocessing) adımlarıyla birleştirir.
- Ham metni doğrudan girip anlaşılır bir yanıt almayı sağlar; iç detayları gizler.
- Pipeline'a metin geçtiğinde 3 ana adım gerçekleşir:
 - Metin, modelin anlayacağı biçime **ön işlenir**.
 - Ön işlenmiş girdiler **modele iletilir**.
 - Modelin tahminleri, yorumlanabilir olması için **son işlenir**.
- Varsayılan olarak göreve uygun, önceden eğitilmiş bir model otomatik seçilir.
- Model, nesne ilk oluşturulduğunda indirilir ve **ön belleğe alınır**; sonraki çalıştırmalarda tekrar indirilmez.
- `pipeline()` metin, görüntü, ses ve çok modlu (multimodal) görevleri destekler.
- Hub'dan belirli bir model seçilebilir: `pipeline("text-generation", model="HuggingFaceTB/SmolLM2-360M")`.

Duygu Analizi (Sentiment Analysis)

- `pipeline("sentiment-analysis")` ile yapılır.
- Varsayılan model İngilizce duygu analizi için ince ayarlıdır.
- Çıktı: `{'label': 'POSITIVE', 'score': 0.96}` biçiminde etiket + güven skoru.
- Tek bir cümle veya **cümle listesi** (batch) verilebilir; liste hâlinde her cümle için ayrı sonuç döner.

Zero-shot Sınıflandırma

- `pipeline("zero-shot-classification")` ile yapılır.
- Etiketsiz metni, **ince ayar gerektirmeden** sınıflandırır.
- `candidate_labels` parametresiyle etiket kümesini kullanıcı belirler (örn. `["education", "politics", "business"]`).
- Modele ait hazır etiketlere bağımlı kalmazsınız; istediğiniz her etiket için olasılık skoru döndürür.
- Etiket açıklamanın (annotation) zaman alıcı olduğu gerçek dünya senaryolarında güçlüdür.

Metin Üretimi (Text Generation)

- `pipeline("text-generation")` ile yapılır.
- Bir istem (prompt) verilir; model kalan metni otomatik tamamlar (tahminli metin benzeri).
- Üretim **rastgelelik içerir**; aynı girdiyle aynı çıktı garanti değildir.
- `num_return_sequences`: kaç farklı dizi üretileceği.
- `max_length` / `min_length`: çıktının toplam uzunluğu.
- Hub'dan belirli bir model (örn. `HuggingFaceTB/SmolLM2-360M`) aynı pipeline'a yüklenebilir.
- Model Hub'daki widget ile model indirilmeden önce çevrimiçi test edilebilir.

Adlandırılmış Varlık Tanıma (NER)

- `pipeline("ner", aggregation_strategy="simple")` ile yapılır.
- Girdi metninde kişi (PER), kuruluş (ORG), konum (LOC) gibi varlıkları bulur.
- Çıktı her varlık için `entity_group`, `score`, `word`, `start`, `end` bilgisi içerir.
- `aggregation_strategy="simple"` aynı varlığa ait kelimeleri birleştirir (örn. "Hugging" + "Face" → tek ORG).
- Ön işlemede kelimeler alt parçalara bölünebilir (örn. `Sylvain` → `S`, `##yl`, `##va`, `##in`); son işleme bunları yeniden gruplar.

Diğer Pipeline'lar (Kısa Bakış)

- `fill-mask: <mask>` belirticini doldurur; `top_k` kaç sonuç gösterileceğini belirler.
- `question-answering`: Bağlamdan bilgi çekerek soruyu yanıtlar (yanıt kendisi üretmez).
- `summarization`: Metni, ana bilgileri koruyarak kısaltır.
- `translation`: Diller arası çeviri (örn. `Helsinki-NLP/opus-mt-fr-en`).
- Görüntü/ses: `image-classification`, `automatic-speech-recognition` gibi.

1/4 · Transformer'lar nasıl çalışır? — Simay Evin

1.1. Önce temel soru: Transformer nedir?

Transformer, metin gibi sıralı verilerdeki parçalar arasındaki ilişkileri **attention (dikkat)** mekanizmasıyla öğrenen bir sinir ağı mimarisidir. 2017'de özellikle makine çevirisi için tanıtıldı; daha sonra BERT, GPT ve T5 gibi birçok model ailesinin temelini oluşturdu.

Terim	Anlamı
Mimari (Architecture)	Modelin iskeleti ve katmanlarının nasıl düzenlendiğidir.
Model	Belirli bir mimariye göre kurulup eğitilmiş sistemdir.
Örnek	Transformer bir mimari ailesidir; BERT, GPT ve T5 bu fikri farklı biçimlerde kullanan model aileleridir.

1.2. Neden Transformer'a ihtiyaç duyuldu?

Transformer'dan önce **RNN** ve **LSTM** gibi yapılar metni büyük ölçüde adım adım işlerdi. Bir zaman adımı önceki adımdan gelen bilgiye bağlı olduğu için uzun dizilerde eğitim sırasında **parallelleştirme** sınırlanır.

Transformer ise tokenlar arasındaki ilişkileri **Self-Attention** ile büyük matris işlemleri halinde hesaplayabilir. Bu nedenle özellikle eğitim aşamasında GPU gibi paralel işlem donanımlarından daha iyi yararlanır.

RNN / LSTM	Transformer
Token 1 → Token 2 → Token 3 → Token 4	Tokenlar arasındaki ilişkiler attention hesapları içinde birlikte değerlendirilebilir.
İşlem sıralı bağımlılığa sahiptir.	Eğitimde hesaplamalar daha iyi paralelleştirilebilir.

Kısa fikir: RNN/LSTM daha çok **adım adım**, Transformer ise eğitim sırasında token ilişkilerini **birlikte hesaplamaya daha uygun** çalışır.

1.3. Transformer modelleri dili nasıl öğrenir?

Transformer modelleri genellikle büyük miktarda ham metin üzerinde bir **language model** olarak **pretraining** görür.

Bu eğitim çoğu zaman **self-supervised learning** şeklindedir. Yani modele insanlar tarafından tek tek hazırlanmış etiketler vermek yerine, öğrenilecek hedef verinin kendi içinden oluşturulur.

Causal Language Modeling

Model önceki tokenlara bakarak sıradaki tokenı tahmin eder.

Örnek:

Girdi: "Bugün hava çok ..."
Tahmin: "güzel"

Masked Language Modeling

Cümlede bazı tokenlar gizlenir ve model eksik tokenı çevresindeki bağlamdan tahmin eder.

Örnek:

Girdi: "Kedi [MASK] içti."
Tahmin: "süt"

Pretraining sonrasında model belirli bir işe uyarlanmak için **fine-tuning / transfer learning** sürecinden geçirilebilir. Böylece dili sıfırdan öğrenmek yerine önceden öğrenilmiş bilgiler yeni görevde kullanılabilir.

Büyük Transformer modellerinin eğitimi pahalı olduğu için **pretrained ağırlıkların paylaşılması** önemlidir. Böylece her proje için aynı dil bilgisini sıfırdan öğrenmek gerekmez.

1.4. Genel Transformer Yapısı: Encoder ve Decoder

Şimdi Transformer'ın genel mimarisine bakalım.

Orijinal Transformer iki ana bloktan oluşur:

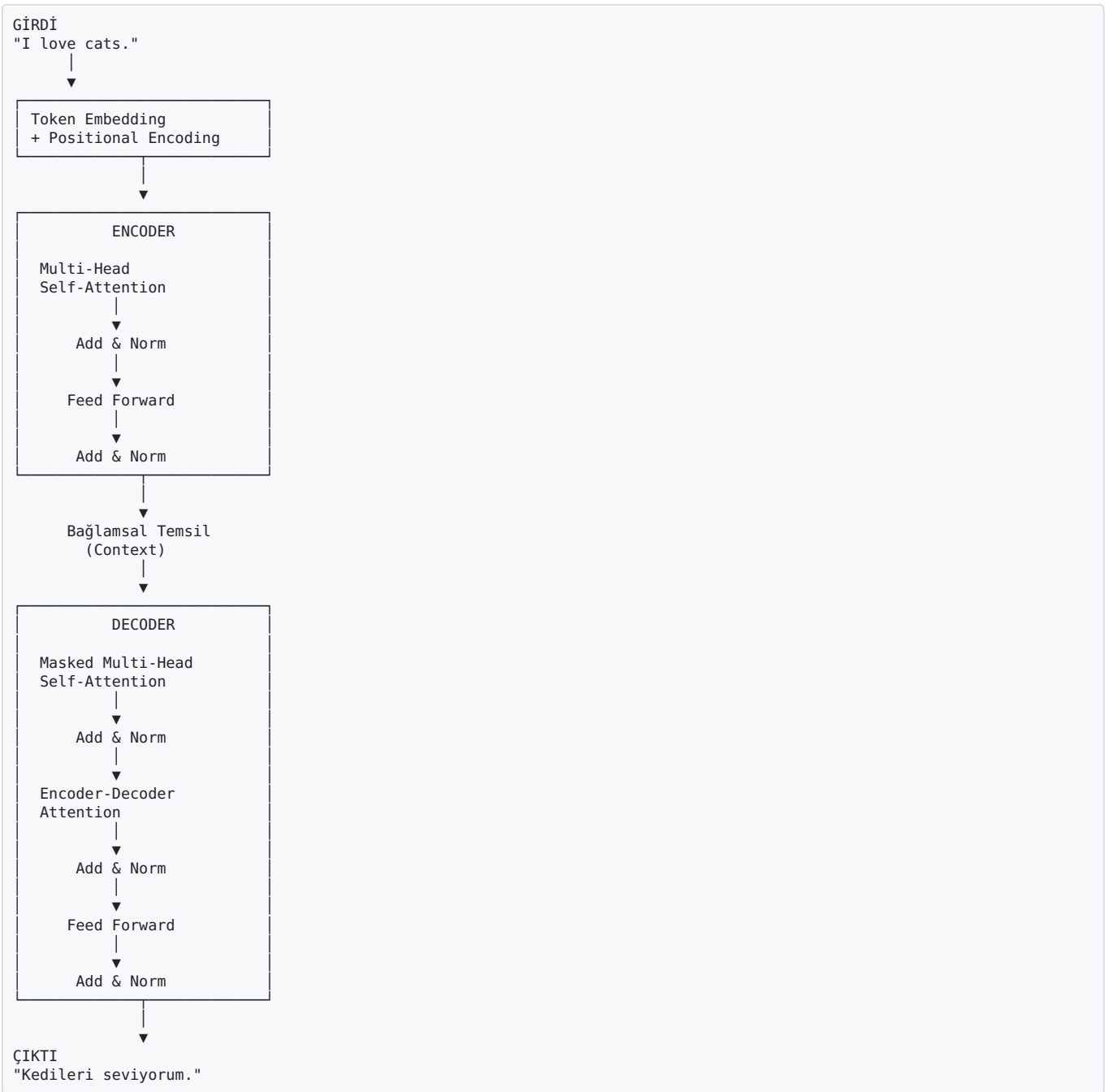
Parça	Görevi
Encoder	Girdiyi okur ve bağlamsal bir temsil oluşturur.
Decoder	Encoder'dan gelen temsili ve daha önce üretilen çıktıları kullanarak hedef diziyi üretir.

Örnek: İngilizceden Türkçeye çeviri

Aşama	Örnek
Girdi	I love cats.
Encoder	Girdinin bağlamını temsil eder.
Decoder	Bu temsilden yararlanarak çıktıyı token token üretir.
Çıktı	Kedileri seviyorum.

1.5. Transformer Blok Şeması

Aşağıdaki şema, Transformer'ın girdiyi nasıl işleyip çıktıya dönüştürdüğünü basitleştirilmiş şekilde gösterir.



Şemadaki kutular ne anlama geliyor?

Blok	Basit açıklama
Token Embedding	Tokenı bilgisayarın işleyebileceği sayısal bir temsile dönüştürür.
Positional Encoding	Tokenın cümlede hangi sırada olduğunu modele bildirir.
Multi-Head Self-Attention	Tokenların birbirleriyle ilişkilerini birden fazla attention başlığı üzerinden değerlendirir.
Feed Forward	Her tokenın temsilini ayrı bir küçük sinir ağıyla işler.
Add & Norm	Bilginin korunmasına ve eğitimin daha kararlı ilerlemesine yardımcı olur.
Masked Self-Attention	Decoder'ın henüz üretilmemiş gelecekteki tokenları görmesini engeller.
Encoder-Decoder Attention	Decoder'ın, encoder tarafından oluşturulan bağlamsal temsilden yararlanmasını sağlar.

1.6. Attention ve Self-Attention Nedir?

Attention, modelin bir tokenı temsil ederken hangi diğer tokenlara daha fazla dikkat etmesi gerektiğini hesaplamasıdır.

Bir kelimenin anlamı bulunduğu bağlama göre değişebildiği için bu ilişki önemlidir.

Örnek

Cümle	Bağlam
“Ali bankaya para yatırdı.”	Buradaki <i>banka</i> , finans kurumudur.
“Ali bankta oturdu.”	Benzer yazılışlı sözcük farklı bir bağlamda farklı anlam taşır.

Model, çevredeki tokenlarla kurduğu ilişkiler sayesinde kelimenin bağlama uygun temsilini oluşturmaya çalışır.

Self-Attention denmesinin nedeni ise bir dizideki tokenların yine **aynı dizideki diğer tokenlarla** ilişkilerinin hesaplanmasıdır.

1.7. Q, K, V ve Self-Attention Formülü

Her tokenın sayısal temsilinden üç farklı temsil üretilir:

Gösterim	Açılımı	Sezgisel anlamı
Q	Query — Sorgu	“Ben hangi bilgiyi arıyorum?”
K	Key — Anahtar	“Ben hangi bilgiyle eşleşebilirim?”
V	Value — Değer	“Ben hangi bilgiyi taşıyorum?”

Q, K ve V üç farklı kelime değildir. Aynı tokenın üç farklı amaç için oluşturulan matematiksel temsilleridir.

Self-Attention formülü:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$$

Formülü adım adım okuyalım

Adım	İşlem	Ne oluyor?
1	QK^T	Query ile Key karşılaştırılır ve tokenlar arasındaki ilişki skorları bulunur.
2	$\div \sqrt{d_k}$	Skorlar ölçeklenir; değerlerin aşırı büyümesi önlenir.
3	softmax	Skorlar dikkat ağırlıklarına dönüştürülür.
4	$\times V$	Dikkat ağırlıkları Value değerleriyle birleştirilir ve bağlama duyarlı yeni temsil oluşur.

$Q \times K^T$
↓
İlişki skorları
↓
 $\div \sqrt{d_k}$
↓
Ölçekleme
↓
 softmax
↓
Dikkat ağırlıkları
↓
 $\times V$
↓
Yeni bağlamsal temsil

Tek cümlelik sezgisel açıklama:

Self-Attention, “**Bu tokenı anlamak için cümledeki hangi tokenlara ne kadar bakmalıyım?**” sorusunun matematiksel cevabını üretir.

1.8. Decoder Neden “Geleceği” Göremez?

Orijinal Transformer’da **encoder**, giriş cümlesinin tamamına bakabilir.

Decoder ise çıktı üretirken henüz üretmediği gelecekteki tokenları göremez. Bu nedenle **Masked Self-Attention** kullanılır.

Örnek

"Bugün hava çok ..."

Decoder sıradaki tokenı üretirken doğru cevabın ilerideki kısmını önceden göremez. Yalnızca:

- daha önceki tokenları,
- encoder’dan gelen bağlamsal temsili

kullanarak sıradaki tokenı tahmin eder.

1.9. Architecture, Checkpoint ve Model

Terim	Anlamı
Architecture	Katmanların ve işlemlerin tanımı; modelin iskeleti.
Checkpoint	Belirli bir mimariye ait eğitilmiş ağırlıklar.
Model	Günlük kullanımda mimariyi ve eğitilmiş sistemi kapsayabilen daha genel terim.

Örnek: BERT bir mimari ailesidir; `bert-base-cased` ise belirli eğitilmiş ağırlıkları olan bir checkpoint olarak düşünülebilir.

1/5 · Transformer'lar görevleri nasıl çözer? — Himmet Can Umutlu

Dil Modeli Eğitiminin İki Ana Yaklaşımı

- Maskelenmiş Dil Modelleme (MLM)** — Encoder (BERT):
 - Girdideki bazı belirteçler rastgele maskelenir ([MASK]).
 - Model, çevreleyen bağlamdan özgün belirteçleri tahmin eder.
 - İki yönlü (bidirectional) bağlam** öğrenir: maskelenen kelimenin hem öncesine hem sonrasına bakar.
- Nedensel Dil Modelleme (CLM)** — Decoder (GPT):
 - Dizideki **tüm önceki belirteçlere** dayanarak bir sonraki belirteci tahmin eder.
 - Yalnızca **soldan (önceki belirteçlerden)** gelen bağlamı kullanabilir.
 - Metin üretiminin temeli: her seferinde sıradaki kelimeyi tahmin eder.

Encoder (BERT) vs Decoder (GPT) Ayrımı

Özellik	Encoder (BERT)	Decoder (GPT)
Bağlam yönü	İki yönlü (sağ + sol)	Tek yönlü (sadece sol)
Eğitim hedefi	MLM + sonraki cümle tahmini	CLM (sonraki token)
Dikkat türü	Tam öz-dikkat	Maskelenmiş öz-dikkat (geleceğe bakamaz)
Tipik görevler	Sınıflandırma, NER, soru cevaplama	Metin üretimi, cümle tamamlama
Belirteçleme	WordPiece	BPE (Bayt Çifti Kodlaması)

BERT (Encoder) Teknik Detayları

- [CLS] : her dizinin başına eklenir; nihai çıktısı sınıflandırma başlığına gider.
- [SEP] : cümleleri ayırmak için kullanılır.
- Parça gömmesi (segment embedding): belirtecin cümle çiftinde 1. mi 2. mi cümlede olduğunu belirtir.
- İki ön eğitim amacı:
 - MLM**: belirli yüzde belirteç maskelenir, model tahmin eder (iki yönlülük "hile"sini çözer).
 - Sonraki cümle tahmini (NSP)**: B cümlesi A'yı takip ediyor mu? (IsNext / NotNext)
- Göreve göre üstüne bir **başlık (head)** eklenir: dizi sınıflandırma, belirteç sınıflandırma, span sınıflandırma.

- Gizli durumlar doğrusal dönüşümle **logits'e** çevrilir; çapraz entropi kaybı hesaplanır.

GPT (Decoder) Teknik Detayları

- BPE ile belirteçleme; her belirtece konumsal kodlama (positional encoding) eklenir.
- Girdi gömmeleri birden çok **decoder bloğundan** geçer.
- Her blokta **maskelenmiş öz-dikkat**: gelecekteki belirteçlere bakılamaz (dikkat maskesiyle skorları 0 yapılır).
- Çıktı, gizli durumları logits'e çeviren **dil modelleme başlığına** gider.
- Etiket, bir sonraki belirteçtir; logits'ler bir adım **sağa kaydırılır** ve çapraz entropi kaybı hesaplanır.
- Ön eğitim amacı tamamen **nedensel dil modellemedir** (sonraki kelimeyi tahmin et).

Üç Mimari Kategori (Özet)

- Encoder-only** (BERT): derin anlama gerektiren görevler (sınıflandırma, NER, QA).
- Decoder-only** (GPT, Llama): metin üretimi, kod üretimi.
- Encoder-Decoder** (T5, BART): diziden diziye görevler (çeviri, özetleme).
- Seçim kuralı: iki yönlü bağlam → encoder; üretim → decoder; dizi→dizi → encoder-decoder.

BART (Encoder-Decoder) Notu

- Kodlayıcısı BERT'e benzer; girdiyi **bozar** ve kod çözücüyle yeniden oluşturur.
- En iyi bozma stratejisi **metin doldurma (text infilling)**: bir span tek [mask] ile değiştirilir.
- Kodlayıcı çıktısı, kod çözücüye ek bağlam sağlar; kod çözücü otoregresif üretir.

1/6 · Transformer Mimarileri — Simay Evin

2.1. Neden Birden Fazla Transformer Mimarisi Var?

Farklı NLP problemleri farklı özelliklere ihtiyaç duyar. Bu nedenle Transformer'ın **encoder** ve **decoder** parçaları üç temel biçimde kullanılır:

Mimari	Ne kullanır?	En uygun olduğu iş	Örnek
Encoder-only	Sadece Encoder	Metni anlama / sınıflandırma	BERT
Decoder-only	Sadece Decoder	Metin üretme	GPT
Encoder-Decoder	İkisi birlikte	Bir diziye başka bir diziye dönüştürme	T5 / BART

2.2. Encoder-only Modeller — BERT Tipi

Encoder-only model yalnızca Transformer'ın **encoder** tarafını kullanır.

Attention katmanı girişteki tüm tokenlara erişebilir. Yani bir tokenın hem solundaki hem sağındaki bağlam kullanılabilir. Buna **bidirectional (çift yönlü) bağlam** denir.

Örnek görev

Aşama	Örnek
Girdi	Bu film hiç güzel değildi.
Amaç	Yorum olumlu mu, olumsuz mu?
Gereken şey	Yeni paragraf üretmek değil, bütün cümleyi doğru anlamak.
Uygun mimari	Encoder-only

BERT tipi modellerin pretraining'i çoğunlukla **Masked Language Modeling** mantığına dayanır: bazı tokenlar gizlenir ve model bağlamdan onları tahmin etmeye çalışır.

Uygun görevler:

- Metin sınıflandırma
- Named Entity Recognition (NER)

- Token sınıflandırma
- Extractive Question Answering

Örnek modeller: BERT, DistilBERT, ModernBERT

2.3. Decoder-only Modeller — GPT Tipi

Decoder-only model yalnızca Transformer'ın **decoder** tarafını kullanır.

Bir tokeni üretirken yalnızca daha önceki tokenlara erişir. Bu nedenle **auto-regressive / causal** yapı olarak anılır.

Örnek

Aşama	Örnek
Girdi / Prompt	Bir robot okula ilk kez gitti ve...
Modelin yaptığı	Bir sonraki tokeni tahmin eder.
Devamı	Ürettiği tokeni bağlama ekler ve sonraki tokeni tahmin eder.
Çıktı	Yeni bir metin dizisi

Basitleştirilmiş üretim:

```
"Bir robot"  
↓  
"Bir robot okula"  
↓  
"Bir robot okula ilk"  
↓  
"Bir robot okula ilk kez"  
↓  
...
```

Modern büyük dil modellerinin çoğu decoder-only mimariyi kullanır.

Tipik süreç:

```
Büyük metin verisi  
↓  
Next-token prediction ile pretraining  
↓  
Instruction tuning / fine-tuning  
↓  
Talimatları daha iyi takip eden model
```

Uygun görevler:

- Metin üretimi
- Sohbet
- Kod üretimi
- Yaratıcı yazma
- Generative Question Answering

Örnek aileler: GPT, Llama, Gemma ve benzeri decoder-only LLM'ler

2.4. Encoder-Decoder Modeller — T5 Tipi

Encoder-decoder veya **sequence-to-sequence** modeller iki tarafı birlikte kullanır.

- **Encoder** girişin tamamını anlamlandırır.
- **Decoder** encoder'ın bağlamsal temsiliyi kullanarak yeni bir çıktı dizisi üretir.

Örnek: Çeviri

Aşama	Örnek
Girdi	I love cats.
Encoder	İngilizce girdinin bağlamını çıkarır.
Decoder	Bu bilgiyi kullanarak Türkçe çıktıyı sırayla üretir.
Çıktı	Kedileri seviyorum.

T5 gibi modellerde pretraining sırasında girişin bazı bölümleri bozulup veya gizlenip modelden eksik kısmı yeniden oluşturması istenebilir. Böylece model hem girdiyi anlamayı hem de çıktı üretmeyi öğrenir.

Uygun görevler:

- Makine çevirisi
- Özetleme
- Grammar correction
- Data-to-text
- Generative Question Answering

Örnek modeller: T5, BART, mBART, Marian

2.5. Hangi Görevde Hangi Mimariyi Seçeriz?

Soru	Tercih
Elimdeki metni anlamam / sınıflandırmam mı gerekiyor?	Encoder-only
Yeni bir metin üretmem mi gerekiyor?	Decoder-only
Bir giriş dizisini başka bir çıktı dizisine dönüştürmem mi gerekiyor?	Encoder-Decoder

Hızlı hafıza kuralı:

BERT = ANLA

GPT = ÜRET

T5 = ANLA + DÖNÜŞTÜR / ÜRET

2.6. Uzun Dizilerde Attention Maliyeti

Standart **full attention** yapısında her token birçok diğer tokenla ilişki kurduğu için attention matrisi dizi uzadıkça hızla büyür.

Standart attention hesaplamasının maliyeti yaklaşık **$O(n^2)$** düzeyindedir.

Bu nedenle çok uzun metinlerde daha verimli özel attention yöntemleri geliştirilmiştir.

Yöntem	Temel fikir
Longformer — Local Attention	Her token bütün diziye değil, çoğunlukla yakınındaki bir pencereye bakar.
Reformer — LSH Attention	Her Query için en ilgili Key'leri daha verimli bulmayı amaçlar.
Axial Positional Encodings	Çok uzun dizilerde konum bilgisini daha verimli tutmayı hedefler.

Ana fikir: **Uzun dizilerde attention maliyetini azaltmak.**

2.7. Genel Özet

Transformer tek bir sabit model değildir. Encoder ve decoder parçalarının nasıl kullanıldığına göre farklı mimari aileleri oluşur.

Model ailesi	Mimari	Temel amaç
BERT tipi	Encoder-only	Metni anlamak
GPT tipi	Decoder-only	Metin üretmek
T5 tipi	Encoder-Decoder	Girdiyi anlayıp yeni bir çıktı dizisine dönüştürmek

Temel Teknik Noktalar

- Self-Attention ve Q-K-V mantığı**
- $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$ formülünün sezgisi
- Transformer'ın RNN/LSTM'ye göre **eğitimde paralelleştirme avantajı**
- Encoder-only / Decoder-only / Encoder-Decoder** farkları ve **BERT / GPT / T5** örnekleri

Ana kaynaklar: Hugging Face LLM Course — Chapter 1/4 “How do Transformers work?” ve Chapter 1/6 “Transformer Architectures”.

1/7 · Kısa sınav — Sıla Taşan

1/7 sorularının Türkçe hâli, özgün akademik sorular ve çözüm anahtarı [soru-cevap/2026-guz-nlp01.md](#) dosyasındadır.

1/8 · LLM'lerle çıkarım — Himmet Can Umutlu

Temel Kavramlar

- Çıkarım (inference):** eğitilmiş LLM'in verilen istemden (prompt) insan benzeri metin üretme süreci.
- Model, yanıtı **her seferinde bir token** üretir; milyarlarca parametreden öğrendiği olasılıklara dayanır.
- Dikkat (attention):** bir sonraki token'ı tahmin ederken en ilgili kelimelere odaklanma yeteneği.
- Bağlam uzunluğu:** modelin bir seferde işleyebildiği maksimum token sayısı ("dikkat süresi"/çalışma belleği).
 - Mimari, hesaplama kaynakları, girdi/çıkıtkarmaşıklığı ile sınırlıdır.

İki Aşamalı Çıkarım Süreci

1. Prefill (Ön Doldurma) Aşaması

- "Yanıt yazmadan önce paragrafın tamamını okumak" gibidir.
- Üç adım:
 - Tokenizasyon:** girdi metnini token'lara dönüştürme.
 - Gömme (embedding):** token'ları anlam taşıyan sayısal vektörlere çevirme.
 - Başlangıç işleme:** gömme vektörlerini modelin sinir ağlarından geçirerek bağlamı anlama.
- Hesaplama yoğun:** tüm girdi token'ları aynı anda (paralel) işlenir.
- İlk token gecikmesini (TTFT) büyük ölçüde bu aşama belirler.

2. Decode (Kod Çözme / Üretim) Aşaması

- Gerçek metin üretiminin gerçekleştiği yer.
- Otoregresif (autoregressive):** her yeni token önceki tüm token'lara bağlıdır.
- Her yeni token için tekrarlanan adımlar:
 - Dikkat hesaplaması:** önceki token'lara geri bakma.
 - Olasılık hesaplaması:** her olası sonraki token'ın olabilirliğini bulma.
 - Token seçimi:** olasılıklara göre bir sonraki token'ı seçme.
 - Devam kontrolü:** üretmeye devam mı, durma mı (EOS)?
- Bellek yoğun:** daha önce üretilen tüm token'lar ve ilişkileri takip edilir.

Örnekleme Stratejileri (Token Seçimi)

- Model, sözlükteki her kelime için **ham logits** (işlenmemiş olasılıklar) üretir.

Temperature (Sıcaklık)

- "Yaratıcılık kadranı": olasılık dağılımını daraltır/genişletir.
- > **1.0**: daha rastgele, yaratıcı, çeşitli seçimler.
- < **1.0**: daha odaklı ve deterministik (keskin dağılım).
- **Greedy Search (açgözlü arama)**: her adımda en yüksek olasılıklı token'ı (argmax) seçme; sıcaklığın en deterministik uç noktası. Hızlı ama tekrara ve sığ sonuçlara yatkındır.

Top-k Filtreleme

- Yalnızca **en olası k** sonraki token dikkate alınır; kalanı elenir.
- Dağılım bu k token'a göre yeniden normalize edilir.

Top-p (Nucleus) Örnekleme

- Sabit sayı yerine, kümülatif olasılığı bir eşiğe (örn. %90) ulaşana kadar en olası kelimeler seçilir.
- Top-k'ya göre dağılımın şekline daha uyumludur; dinamik aday kümesi oluşturur.

Tekrarı Yönetmek (Penaltılar)

- **Varlık cezası (presence penalty)**: daha önce görünmüş her token'a, sıklığına bakmaksızın **sabit** ceza.
- **Sıklık cezası (frequency penalty)**: bir token ne kadar çok kullanıldıysa ceza o kadar **artar**.
- Bu cezalar, diğer örnekleme stratejilerinden **önce** ham logits'lere uygulanır.

Üretim Uzunluğunu Kontrol Etmek

- **Token sınırları**: min/maks token sayısı belirleme.
- **Durdurma dizileri**: üretim sonunu işaretleyen desenler (örn. `"\n\n"`).
- **EOS tespiti**: modelin yanıtı doğal yoldan bitirmesine izin verme (SmolLM2'de `<|im_end|>`).

Beam Search (Hüzme Araması)

- Tek tek token kararı yerine **birden fazla aday yolu aynı anda** keşfeder (satranç gibi).
- Adımlar:
 1. Her adımda **birden fazla aday dizi** tut (tipik 5-10).
 2. Her aday için sonraki token olasılıklarını hesapla.
 3. En umut verici dizi + token kombinasyonlarını sakla.
 4. İstenen uzunluğa / durma koşuluna kadar devam et.
 5. **En yüksek toplam olasılıklı** diziyi seç.
- Daha tutarlı ve dilbilgisel olarak doğru metin üretir, ancak **daha fazla hesaplama** gerektirir.

Pratik Zorluklar ve Optimizasyon

- **Performans metrikleri**:
 - TTFT (ilk token'a kadar geçen süre) — prefill'den etkilenir.
 - TPOT (çıktı token'ı başına süre) — üretim hızını belirler.
 - Throughput (verim) — aynı anda kaç istek.
 - VRAM kullanımı — genellikle birincil kısıt.
- **Bağlam uzunluğu maliyeti**:
 - Bellek kullanımı $\propto$ uzunluk² (kuadratik).
 - İşlem süresi $\propto$ uzunluk (doğrusal).
- **KV Cache (Anahtar-Değer Önbellediği)**: ara hesaplamaları depolayıp yeniden kullanır; tekrarlı hesabı azaltır, üretimi hızlandırır (bedeli ek bellek).

1/9 · Önyargı ve sınırlamalar — Abdulkadir Öcal

1. Önceden Eğitilmiş Modellerin Doğası ve Veri Kaynağı

- **Ham İnternet Verisi**: Büyük dil modelleri (BERT, GPT vb.), internet üzerinden taranmış devasa ham metin yığınlarıyla (web kazıma, forumlar, haber siteleri) ön eğitime (pre-training) tabi tutulur.
- **Verinin Aynası Olma**: Modeller dünyayı insan gibi algılamaz veya tarafsız bir ahlak mekanizmasına sahip değildir. İnternetteki insan kaynaklı önyargılar, ırkçılık, cinsiyetçilik ve mesleki kalıp yargılar (stereotipler) doğrudan modelin olasılık dağılımına aktarılır.

2. İstatistiki Önyargı (Bias) Nasıl Ortaya Çıkar?

- Mask Filling Örneği:** Kurs dokümanı fill-mask pipeline'ı üzerinden BERT'e "This man works as a [MASK]" ve "This woman works as a [MASK]" cümlelerini verdiğinde; erkeğe "lawyer, engineer, doctor", kadına ise "waitress, nurse, teacher" gibi kalıp mesleklerin en yüksek olasılıkla atandığını gösterir.
- İnce Ayar (Fine-Tuning) Yanılsaması:** Dokümanın en kritik uyarısı şudur: "Modeli kendi özel verinizle fine-tune etmek, modelin temel mimarisindeki ve ön eğitimindeki bu kök önyargıları tamamen yok etmez; sadece üstünü örter."

3. Üretim (Production) Ortamı Riskleri ve Sorumluluk

- Gerçek Dünya Tehlikesi:** Bir dil modelini doğrudan filtrelemeden müşteri hizmetlerine veya ürün öneri sistemine bağlamak; modelin nefret söylemi, ayrımcılık veya kendinden emin halüsinasyonlar üretmesine yol açabilir.
- Geliştiricinin Görevi:** NLP mühendisinin görevi sadece model eğitmek değil; bu modelleri canlıya almadan önce adli bilişim (red-teaming) testlerinden geçirmek ve sınırlarını raporlamaktır.

Bölüm 1.9 ile Notebook Deneylerinin Eşleşmesi

Bölüm 1.9 Teorik Kuralı	Notebook'ta Kanıtlanan Deney
Cinsiyet / Kalıp Yargı (Stereotype)	BERT mask-filling ile meslek/cinsiyet softmax olasılık karşılaştırması
Doğruluk Sınırı / Yanılsama	GPT-2'ye tarihte olmayan olay sorarak üretilen karınca algoritması halüsinasyonu
Güvenlik / Kontrol Edilebilirlik	Prompt Injection ile sistem kuralının bypass edilmesi denemesi
Mimari / Bellek Sınırı (Attention)	truncation=False ile 1024 token limitinin aşılıp IndexError patlatılması

1/10 · Ünite Özeti — Özge Sayınbaş

Kaynak: huggingface.co/learn/llm-course/chapter1/10

Doğal Dil İşleme ve LLM'ler: NLP, metni sınıflandırmaktan yeni metin üretmeye kadar çok geniş bir görev yelpazesini kapsar. LLM'ler, devasa miktarda metinle eğitilmiş güçlü modellerdir ve tek bir mimari içinde birden fazla görevi yerine getirebilir. Bu yeteneklerine rağmen halüsinasyon ve önyargı gibi sınırlamaları vardır.

Örnek: Aynı LLM'ye önce bir yorumun olumlu mu olumsuz mu olduğunu sorup ardından bir metni özetletebilirsiniz; iki iş için ayrı model gerekmez. Ancak model bazen yanlış bir bilgiyi doğruymuş gibi söyleyebilir (halüsinasyon) ya da eğitim verisindeki önyargıları tekrarlayabilir.

Transformer modelleri, günümüz NLP'sinin ve LLM'lerin temelini oluşturur. Hugging Face'in pipeline() fonksiyonu, ön eğitilmiş bu modelleri metin sınıflandırma, soru yanıtlama, metin üretimi, özetleme, çeviri, konuşma tanıma ve görüntü sınıflandırma gibi görevlerde kolayca kullanmayı sağlar.

Örnek:

```
from transformers import pipeline
classifier = pipeline("sentiment-analysis")
classifier("I've been waiting for a HuggingFace course my whole life.")
# [{'label': 'POSITIVE', 'score': 0.9598047137260437}]
```

Birkaç satır kodla hazır bir model yüklenir ve cümlelerin olumlu olduğu bulunur.

Transformer'ların temelinde **dikkat (attention) mekanizması** vardır: Model, bir kelimeyi işlerken cümledeki ilgili kelimelere odaklanır.

Örnek: "Fransa'nın başkenti ..." cümlesinde model, sonraki kelimenin "Paris" olduğunu tahmin etmek için en çok "Fransa" ve "başkent" kelimelerine dikkat eder.

Transfer öğrenme (transfer learning) sayesinde, büyük veriyle önceden eğitilmiş bir model daha az veriyle yeni bir

göreve uyarlanabilir.

Örnek: İngilizce metinlerle önceden eğitilmiş bir model, bilimsel makalelerle kısa bir ek eğitimden geçirilerek bilim alanında uzmanlaşmış bir modele dönüştürülebilir. Sıfırdan eğitmeye göre çok daha az zaman, veri ve maliyet gerekir.

Transformer modelleri üç ana türe ayrılır:

Tür	Ne yapar?	Örnek model	Örnek görev
Encoder	Metni anlar	BERT	Bir yorumun olumlu mu olumsuz mu olduğunu bulmak
Decoder	Metin üretir	GPT, LLaMA	Bir hikâyenin devamını yazmak, sohbet etmek
Encoder-decoder	Metni başka bir metne dönüştürür	BART, T5	Bir metni çevirmek veya özetlemek

Modern LLM gelişmeleri: Modeller boyut ve yetenek bakımından giderek büyümektedir; bu büyümeye ölçekleme yasaları (scaling laws) yön verir. Modern LLM'ler iki aşamada eğitilir: önce çok büyük veriyle ön eğitim (pretraining), ardından talimatlara uymak üzere talimat ayarı (instruction tuning). Daha uzun metinleri işleyebilmek için özelleşmiş dikkat mekanizmaları geliştirilmiştir.

Pratik uygulamalar: Ön eğitilmiş modeller Hugging Face Hub'dan bulunup kullanılabilir, Inference API ile doğrudan tarayıcıda denenebilir. Önemli olan, göreve en uygun modeli seçmektir.

Örnek: Metin üretecek bir model arıyorsanız Hub'da "text-generation" etiketine tıklayıp listelenen modellerden birini seçebilir, indirmeden önce sayfasındaki bileşenle (widget) deneyebilirsiniz.

1/11 · Sertifika sınavı — Sıla Taşan

1/11 sertifikasyon sınavına ilişkin çalışma [soru-cevap/2026-guz-nlp01.md](#) dosyasındadır.